

ttx 工具详解笔记

本文档基于 **fontTools 4.61.0** (**ttx** 命令行工具) 实际演示整理。
所有终端输出均为**真实运行结果**逐字拷贝，演示字体见[附录 B](#)。

目录

- [第一部分 认识 ttx](#)
- [第二部分 ttx --help 手册全文 + 逐条中文详解](#)
- [第三部分 字体文件结构基础](#)
- [第四部分 Dump 操作实战](#)
- [第五部分 Compile 操作实战](#)
- [第六部分 TTX \(XML\) 格式逐表解读](#)
- [第七部分 字体特性与功能](#)
- [第八部分 常见工作流](#)
- [附录 A 命令行选项速查表](#)
- [附录 B 演示字体清单](#)
- [附录 C 常见表标签速查](#)

第一部分 认识 ttx

1.1 ttx 是什么

ttx 是开源字体工具包 **fontTools** 提供的命令行工具，用于在 **二进制字体文件** (OpenType / TrueType) 和 **TTX** (一种基于 XML 的文本格式) 之间做**双向转换**：



- **输入是字体** (`.ttf` / `.otf` / `.ttc` / `.otc` / `.woff` / `.woff2`) → 反编译成 `.ttx` (XML 文本)。
- **输入是** `.ttx` → 编译回二进制字体 (TrueType 或 OpenType/CFF，取决于数据本身)。
- 特殊输入值 `-` 表示从**标准输入**读取。

为什么有用？ 字体二进制是紧凑但不可读的结构化格式，**TTX 把每张“表”(table) 展开成人类可读、可用文本编辑器/diff/Git 管理的 XML**，使得**检视、修改、比对、版本控制**字体成为可能。修改 XML 后再用 **ttx** 编译回去，即可得到新字体。

1.2 安装与环境

`ttx` 随 `fontTools` 一起安装：

```
1 pip install fonttools # 基础安装
2 pip install fonttools[woff] # 额外装 zopfli, 支持 --with-zopfli
3 pip install brotli # 额外装 brotli, 支持 WOFF2
```

查看版本:

```
1 $ ttx --version
2 4.61.0
```

本文档对应 fontTools 4.61.0。

1.3 两种工作模式

模式	触发条件	作用
Dump (反编译)	输入是字体文件	字体 → TTX (XML)
Compile (编译)	输入是 .ttx 文件	TTX (XML) → 字体

安全设计: ttx 生成的输出文件永不覆盖同名已有文件, 而是自动追加序号 (如 times#1.ttx)。需要覆盖时加 `-f`。

第二部分 ttx --help 手册全文 + 逐条中文详解

2.1 帮助原文 (逐字拷贝)

注: `ttx --help` 并非有效选项 (实际触发帮助的是 `-h`) , ttx 会先打印帮助再报 `option -help not recognized` 退出。以下为完整输出:

```
1 usage: ttx [options] inputfile1 [... inputfileN]
2
3 TTX -- From OpenType To XML And Back
4
5 If an input file is a TrueType or OpenType font file, it will be
6 decompiled to a TTX file (an XML-based text format).
7 If an input file is a TTX file, it will be compiled to whatever
8 format the data is in, a TrueType or OpenType/CFF font file.
9 A special input value of - means read from the standard input.
10
11 Output files are created so they are unique: an existing file is
12 never overwritten.
13
14 General options
15 =====
16
17 -h Help          print this message.
18 --version        show version and exit.
19 -d <outputfolder> Specify a directory where the output files are
20                  to be created.
```

```

21 -o <outputfile>    Specify a file to write the output to. A special
22                    value of - would use the standard output.
23 -f                Overwrite existing output file(s), ie. don't append
24                    numbers.
25 -v                Verbose: more messages will be written to stdout
26                    about what is being done.
27 -q                Quiet: No messages will be written to stdout about
28                    what is being done.
29 -a                allow virtual glyphs ID's on compile or decompile.
30
31 Dump options
32 =====
33
34 -l                List table info: instead of dumping to a TTX file, list
35                    some minimal info about each table.
36 -t <table>        Specify a table to dump. Multiple -t options
37                    are allowed. When no -t option is specified, all tables
38                    will be dumped.
39 -x <table>        Specify a table to exclude from the dump. Multiple -x
40                    options are allowed. -t and -x are mutually exclusive.
41 -s                Split tables: save the TTX data into separate TTX files per
42                    table and write one small TTX file that contains references
43                    to the individual table dumps. This file can be used as
44                    input to ttx, as long as the table files are in the
45                    same directory.
46 -g                Split glyph table: Save the glyph data into separate TTX files
47                    per glyph and write a small TTX for the glyph table which
48                    contains references to the individual TTGlyph elements.
49                    NOTE: specifying -g implies -s (no need for -s together
50                    with -g)
51 -i                Do NOT disassemble TT instructions: when this option is
52                    given, all TrueType programs (glyph programs, the font
53                    program and the pre-program) will be written to the TTX
54                    file as hex data instead of assembly. This saves some time
55                    and makes the TTX file smaller.
56 -z <format>        Specify a bitmap data export option for EBDT:
57                    {'raw', 'row', 'bitwise', 'extfile'} or for the CBDT:
58                    {'raw', 'extfile'} Each option does one of the following:
59
60                    -z raw
61                        export the bitmap data as a hex dump
62                    -z row
63                        export each row as hex data
64                    -z bitwise
65                        export each row as binary in an ASCII art style
66                    -z extfile
67                        export the data as external files with XML references
68
69                    If no export format is specified 'raw' format is used.
70 -e                Don't ignore decompilation errors, but show a full traceback
71                    and abort.
72 -y <number>        Select font number for TrueType Collection (.ttc/.otc),
73                    starting from 0.
74 --unicodedata <UnicodeData.txt>
75                    Use custom database file to write character names in the
76                    comments of the cmap TTX output.

```

```

77 --newline <value>
78         Control how line endings are written in the XML file. It
79         can be 'LF', 'CR', or 'CRLF'. If not specified, the
80         default platform-specific line endings are used.
81
82 Compile options
83 =====
84
85 -m         Merge with TrueType-input-file: specify a TrueType or
86         OpenType font file to be merged with the TTX file. This
87         option is only valid when at most one TTX file is specified.
88 -b         Don't recalc glyph bounding boxes: use the values in the
89         TTX file as-is.
90 --recalc-timestamp
91         Set font 'modified' timestamp to current time.
92         By default, the modification time of the TTX file will be
93         used.
94 --no-recalc-timestamp
95         Keep the original font 'modified' timestamp.
96 --flavor <type>
97         Specify flavor of output font file. May be 'woff' or 'woff2'.
98         Note that WOFF2 requires the Brotli Python extension,
99         available at https://github.com/google/brotli
100 --with-zopfli
101         Use Zopfli instead of Zlib to compress WOFF. The Python
102         extension is available at https://pypi.python.org/pypi/zopfli
103 --optimize-font-speed
104         Enable optimizations that prioritize speed over file size.
105         This mainly affects how glyf table and gvar / VARC tables are
106         compiled. The produced fonts will be larger, but rendering
107         performance will be improved with HarfBuzz and other text
108         layout engines.

```

2.2 通用选项（General options）逐条详解

选项	中文释义	说明
<code>-h</code>	打印帮助信息	真正的帮助触发符； <code>--help</code> 反而会报错
<code>--version</code>	显示版本号并退出	例：4.61.0
<code>-d <目录></code>	指定输出文件 目录	⚠️ 该目录必须事先存在 ，否则报错 <code>The -d option value must be an existing directory</code>
<code>-o <文件></code>	指定输出 文件名 ； <code>-o -</code> 表示输出到 标准输出	⚠️ <code>-o</code> 与 <code>-d</code> 同时出现时， <code>-o</code> 生效、 <code>-d</code> 被忽略，文件按 <code>-o</code> 的值（相对当前目录）写出
<code>-f</code>	覆盖已有输出文件（不追加序号）	默认行为是永不覆盖、自动加 #1 / #2 ...
<code>-v</code>	详细模式：打印更多进度信息	如 <code>Reading 'head' table from disk / Decompile ...</code>

选项	中文释义	说明
<code>-q</code>	静默模式：不打印任何进度信息	适合脚本化批量处理
<code>-a</code>	允许编译/反编译时使用虚拟字形 ID	边缘场景，处理引用了不存在字形 ID 的字体时使用

2.3 Dump 选项（反编译时）逐条详解

选项	中文释义	说明
<code>-l</code>	列表模式 ：不导出 TTX，只列出每张表的标签/校验和/长度/偏移	快速体检字体的第一手段
<code>-t <表></code>	只导出指定的表（可多次 <code>-t</code> ）	不带 <code>-t</code> 时导出全部表
<code>-x <表></code>	排除指定的表（可多次 <code>-x</code> ）	<code>-t</code> 与 <code>-x</code> 互斥 ，不能同时用
<code>-s</code>	按表拆分 ：每张表单独一个 <code>.ttt</code> 文件，另写一个引用它们的小主文件	便于逐表编辑、Git 提交
<code>-g</code>	按字形拆分 glyph ：每个字形单独一个文件	<code>-g</code> 隐含 <code>-s</code> ，无需再写 <code>-s</code>
<code>-i</code>	不反汇编 TrueType 指令，改为十六进制输出	更快、文件更小；默认会把指令反汇编成可读汇编
<code>-z <格式></code>	位图（EBDT/CBDT）导出格式： <code>raw</code> / <code>row</code> / <code>bitwise</code> / <code>extfile</code>	见 4.9 节
<code>-e</code>	不忽略反编译错误，打印完整 traceback 并中止	调试损坏字体时使用
<code>-y <编号></code>	选择 TTC/OTC 字体集中的字体编号（从 0 起）	字体集合 <code>.ttc</code> 含多个字体，必须用 <code>-y</code> 指定
<code>--unicodedata <文件></code>	用自定义 Unicode 数据库，在 cmap 输出注释里写字符名	自定义/扩展 Unicode 版本时使用
<code>--newline <值></code>	控制 XML 行尾： <code>LF</code> / <code>CR</code> / <code>CRLF</code>	不指定则用平台默认

2.4 Compile 选项（编译时）逐条详解

选项	中文释义	说明
<code>-m <字体></code>	把 TTX 文件 合并 进一个已有的字体文件	最多只能指定一个 TTX 文件；用于"只改几张表"的工作流
<code>-b</code>	不重新计算字形包围盒（bbox），直接用 TTX 里的值	默认会重算

选项	中文释义	说明
<code>--recalc-timestamp</code>	把字体 <code>modified</code> 时间戳设为 当前时间	默认用 TTX 文件的修改时间
<code>--no-recalc-timestamp</code>	保留字体 原始 <code>modified</code> 时间戳	
<code>--flavor <类型></code>	输出字体风味: <code>woff</code> 或 <code>woff2</code>	WOFF2 需额外安装 brotli
<code>--with-zopfli</code>	用 Zopfli (而非 Zlib) 压缩 WOFF	压缩率更高但更慢; 需装 zopfli
<code>--optimize-font-speed</code>	优化渲染速度优先于体积	主要影响 glyf / gvar / VARC 的编译方式; 产物更大但 HarfBuzz 等渲染更快

第三部分 字体文件结构基础

3.1 sfnt 容器与"表"(table)

OpenType/TrueType 字体本质是一个 **sfnt 容器**，里面装着一堆带 4 字符标签 (tag) 的**表**(table)。每张表负责字体的一个方面：

- `head` —— 字体头 (版本、unitsPerEm、bbox、时间戳...)
- `name` —— 命名 (字体名、版权、版本字符串...)
- `cmap` —— 字符到字形的映射
- `glyf` / `CFF` —— 字形轮廓数据
- `hhea` / `hmtx` —— 水平度量
- `GSUB` / `GPOS` —— OpenType 布局 (替换 / 定位)
-

3.1.1 sfnt 容器的二进制结构

"sfnt" 本身只是一种**容器 (包装) 格式** (源自 Apple, 原指 "spline font")，规定"如何把若干张表打包进一个文件"——至于里面装的是 TrueType 轮廓还是 CFF 轮廓，容器并不关心。`.ttf` 和 `.otf` 用的是**同一个 sfnt 容器**，区别只在文件头的"魔数"和内部的轮廓表。文件由三段顺序排列：

1. **偏移表 (Offset Table, 12 字节)** —— 容器头，给出表的数量与查找参数；
2. **表目录 (Table Directory)** —— 每张表一个 16 字节条目，告诉你这张表在文件里的位置；
3. **表数据** —— 各张表的实际字节，按 4 字节对齐填充。

偏移表第一个 4 字节是 **sfVersion 魔数**，一眼区分字体类型：

魔数 (十六进制)	ASCII	含义
00 01 00 00	—	TrueType (<code>.ttf</code> , 轮廓在 <code>glyf</code>)
4F 54 54 4F	OTTO	OpenType/CFF (<code>.otf</code> , 轮廓在 <code>CFF</code>)

魔数 (十六进制)	ASCII	含义
74 72 75 65 / 00 74 79 70 31	true / typ1	个别字体用的等价标记

以 `times.ttf` 为例，文件开头的真实字节 (`xxd` 输出)：

```

1 00000000: 0001 0000 0019 0100 0004 0090 4453 4947 .....DSIG
2 00000010: ac40 e597 0012 3028 0000 25ac 4744 4546 .@....0(.,%.GDEF
3 00000020: 240e 2921 000f c010 0000 0340 4750 4f53 $.)!.....@GPOS

```

逐段拆解：

- **偏移表 (前 12 字节 00 01 00 00 | 00 19 | 01 00 | 00 04 | 00 90)：**
 - 00 01 00 00 — sfVersion, TrueType 魔数；
 - 00 19 — numTables = **25** (本字体含 25 张表)；
 - 01 00 / 00 04 / 00 90 — searchRange=256、entrySelector=4、rangeShift=144，是为**二分查找预计算**的参数 ($\text{searchRange} = 2^{\lceil \log_2 N \rceil} \times 16$ 、 $\text{rangeShift} = N \times 16 - \text{searchRange}$)，现代实现基本忽略，留作兼容。
- **表目录 (紧跟其后，每张表 16 字节 = tag 4 + checksum 4 + offset 4 + length 4)：** 第一条是 DSIG——ac40e597 (校验和) 00123028 (偏移=1191976) 000025ac (长度=9644)；第二条 GDEF——240e2921 000fc010 (1032208) 00000340 (832)。tag 是 4 个 ASCII 字符；checksum 用于校验该表数据完整性。

想读取某张表，容器只需"在表目录里按 tag 查到 offset + length，再跳到该偏移读 length 字节"——这正是所有字体工具 (含 `ttx`) 定位表的方式。

与 `ttx -l` 的对应： `ttx -l` 的四列 tag / checksum / length / offset 就是把**表目录的四个字段**原样打印。对照上面的 `times.ttf` 目录——DSIG offset=1191976 length=9644、GDEF offset=1032208 length=832——与 3.2 节的 `ttx -l` 输出逐字一致。这也是 TTX 根元素写作 `<ttFont sfntVersion="\x00\x01\x00\x00">` (TrueType) 或 `sfntVersion="OTTO"` (CFF) 的原因——它记录的正是偏移表里的这个魔数。

`ttx -l` 是查看一个字体"装了哪些表"的最快方式。

3.2 ttx -l 列表实战 (5 个字体对比)

Times New Roman (`times.ttf`, 西文 TrueType)

```

1 $ ttx -l times.ttf
2 Listing table info for "times.ttf":
3     tag      checksum      length      offset
4     ----      -
5     DSIG  0xAC40E597      9644      1191976
6     GDEF  0x240E2921       832      1032208
7     GPOS  0x0340B599     126438      1033040
8     GSUB  0x2267ED52     32366      1159480
9     JSTF  0x6D2A6906       30      1191848
10    LTSH  0xFAFD0F81      4733      19548
11    OS/2  0x176F5876       96        536
12    PCLT  0x59D381E3       54      1032152
13    VDMX  0x4E236882     4500      24284

```

14	cmap	0xE969B938	4510	142360
15	cvt	0x1D6402D7	4228	153512
16	fpgm	0x6D591B53	2649	146872
17	gasp	0x00190021	16	1032136
18	glyf	0x52EA912B	838298	176660
19	hdmx	0x03D7905C	113576	28784
20	head	0xF4B9D98E	54	412
21	hhea	0x134D1A91	36	468
22	hmtx	0x2005A771	18916	632
23	kern	0xA677ACD1	5220	1014960
24	loca	0x7B4F9A90	18920	157740
25	maxp	0x1941135D	32	504
26	meta	0x03A00561	96	1191880
27	name	0x1A5574B8	11924	1020180
28	post	0xFF240064	32	1032104
29	prep	0xB3329291	3987	149524

四列分别是：**表标签**、**校验和**、**字节长度**、**在文件中的偏移**。可以看到这是一个内容丰富的 TrueType 字体：含 `glyf` (轮廓) + `loca` (轮廓索引)、`fpgm` / `prep` / `cvt` (hinting 程序)、`GSUB` / `GPOS` / `GDEF` / `JSTF` (布局)、`hdmx` / `VDMX` / `LTSH` (各种度量优化)。

微软雅黑 (`msyh.ttc`，中文 TTC 集合)

`.ttc` 是字体集合，必须用 `-y` 指定字体编号：

```

1 $ ttx -l msyh.ttc
2 ERROR: specify a font number between 0 and 1 (inclusive)

```

该集合含 **2 个字体** (编号 0、1)。查看编号 0：

```

1 $ ttx -l -y 0 msyh.ttc
2 -----
3 GDEF 0xE09B5667      42      856
4 GPOS 0xEAA2F7BC     1506     900
5 GSUB 0x3161C84A     2436    2408
6 LTSH 0xCCDD6463    30213    4844
7 MERG 0x00160001      12    35060
8 OS/2 0x85B823AD      96    35072
9 VDMX 0x73EC7B66     1504    35168
10 cmap 0x87CA4F2F    103666    36672
11 cvt  0x4C086ADE     1014    140340
12 fpgm 0x2C98B72C     2566    141356
13 gasp 0x001D0023      16    143924
14 glyf 0x6807FEF9   18617410    143940
15 hdmx 0xD4172CA5    513612   18761352
16 head 0x1A075ABF      54   19274964
17 hhea 0x115E7599      36   19275020
18 hmtx 0x8074F0B4    120164   19275056
19 kern 0xCDB4CE62     3516   19395220
20 loca 0x52561FE6    120840   19398736
21 maxp 0x7F510D70      32   19519576
22 meta 0x689BBA3B      48   19519608
23 name 0xE8CA67D9     3150   19519656
24 post 0xFF517639      32   19522808

```


25	prep	0x44BD05CF	754	19522840
26	vhea	0x0ED005F4	36	19523596
27	vmtx	0xD74A0E0F	60420	19523632

注意几个 CJK 字体特征：glyf 高达 **18.6 MB**（几万个汉字轮廓）、cmap **103 KB**（超大字符映射）、含 vhea / vmtx（纵向排版度量）、MERG（字形合并提示）。

Cambria (cambria.ttc, 含数学字体)

编号 1 是 Cambria Math, 独有 MATH 表:

```

1 $ ttx -l -y 1 cambria.ttc
2 ...
3 MATH 0x4F508D8B      25722      1726288
4 ...
5 EBDT 0x0E304C68      22252         744
6 EBLC 0xF2D855C0       5600       22996
7 ...

```

MATH 表 (25 KB) 是 OpenType 数学排版的核心; EBDT / EBLC 是嵌入式点阵位图 (小字号下的预渲染字形)。

思源黑体 (SourceHanSansSC-Regular.otf, CFF 轮廓)

```

1 $ ttx -l SourceHanSansSC-Regular.otf
2 ----
3 BASE 0xEDFAF516      240 15791504
4 CFF  0xC9F11CC0 15551854 239648
5 DSIG 0x00000001      8 15791744
6 GDEF 0x020E0201      28 15791752
7 GPOS 0xF2C8233F     46568 15791780
8 GSUB 0x6C9FFF0C    167048 15838348
9 OS/2 0x92EC11E3      96      384
10 VORG 0xD86AF415      920 16005396
11 cmap 0x57B9899D    236797      2816
12 head 0x2F8E7185      54      284
13 hhea 0x0C12084D      36      340
14 hmtx 0xAB3609B4    262068 16006316
15 maxp 0xFFFF5000       6      376
16 name 0xFF97DD0C     2336      480
17 post 0xFF860074      32     239616
18 vhea 0x0CB215B2      36 16268384
19 vmtx 0xF9498662    261412 16268420

```

这是 **CID-keyed CFF** 字体 (PostScript 轮廓): 用 CFF 表 (15.5 MB) 而非 glyf; maxp 只有 **6 字节** (CFF 字体的 maxp 极简); 含 VORG (纵向原点)、BASE (基线表)、vhea / vmtx (CJK 纵向)、超大 GSUB (167 KB)。

思源宋体可变版 (NotoSerifCJKsc-VF.ttf, 可变 TrueType)

```
1 $ ttx -l NotoSerifCJKsc-VF.ttf
2      ----  -----  -----  -----
3      BASE  0x711D8E91      278  35032040
4      GDEF  0x0FD3FF04      202  35293520
5      GPOS  0x9767C28C     121036  35293724
6      GSUB  0x3847F4A4     163842  35414760
7      HVAR  0xD4DA58D3     63253  35578604
8      OS/2  0x9EFB1491       96    472
9      STAT  0x80E79632      182  35641860
10     avar  0x79F58926       42  35642044
11     cmap  0x851F7BE2     234521  262424
12     fvar  0x8D4C75DC      106  35642088
13     glyf  0xCCA2ECA7    34269540  759092
14     gvar  0x032BB0A0    24257464  35642196
15     head  0x2C3A7C61       54    348
16     hhea  0x0C160765       36    404
17     hmtx  0x46FC1B45     261854  568
18     loca  0xC4145630     262144  496948
19     maxp  0x0066041C       32    440
20     name  0x46EA21BD     3373  35028632
21     post  0xFFB80032       32  35032008
22     vhea  0x0CBF1564       36  59899660
23     vmtx  0x218657EB     261198  35032320
```

这是一个**可变字体** (Variable Font)。注意它用的是 **TrueType (glyf) 轮廓**而非 CFF——可变版 Noto 与上面的静态思源黑体轮廓格式不同。比静态字体多了 5 张可变字体专属表：**fvar** (轴定义)、**STAT** (轴值命名)、**avar** (轴映射)、**gvar** (字形差量)、**HVAR** (度量差量)。其中 **gvar** (24 MB) 是全字体最大表，承载 6 万多字形的形变差量；这些表的逐表解读见 [6.15 节](#)。

3.3 TTC/OTC 字体集合：枚举各 index

3.3.1 什么是 TTC/OTC 字体集合

TTC (TrueType Collection) 和 **OTC** (OpenType Collection) 是"字体集合"容器：一个文件里打包**多个字体**。两者容器格式相同 (文件头 4 字节都是 **ttcf**)，区别只在内部轮廓——**.ttc** 装 TrueType (**glyf**) 字体，**.otc** 装 CFF (**CFF**) 字体。

集合最大的价值是**表去重**：多个字体若共用同一张表 (如同一套 **glyf** 轮廓)，文件里只存一份、靠偏移引用。例如 **msyh.ttc** (微软雅黑 + 雅黑 UI) 的 25 张表里有 **19 张完全共享**；**cambria.ttc** (Cambria + Cambria Math) 共享 13 张、含 **glyf/loca/hmtx**，只有 Math 字体独占一张 **MATH**。

对 ttx 而言：TTC/OTC 必须用 **-y <编号>** 选定其中一个字体再操作 (见第 2.3 节的 **-y**)。**-y** 从 0 起。

3.3.2 发现集合里有几个字体 (index 数量)

ttx -l 不带 **-y** 直接打 TTC 会报错——但报错信息**透露了 index 范围**，这是最快的"数数"方法：

```
1 $ ttx -l Songti.ttc
2 ERROR: specify a font number between 0 and 7 (inclusive)
```

→ 0 到 7，共 **8 个字体**。或者用 fontTools 一行直接拿到（无需猜测）：

```
1 $ python -c "from fontTools.ttLib import TTCollection;
  print(len(TTCollection('Songti.ttc').fonts))"
2 8
```

3.3.3 逐个 index 列出表信息（-y 迭代）

知道数量后，用 `-y N` 逐个 dump 列表，即可对比每个字体装了哪些表：

```
1 $ ttx -l -y 0 Songti.ttc
2 Listing table info for "Songti.ttc":
3     tag      checksum      length      offset
4     ----      -
5     OS/2     0x652F6BDA      96      2628
6     ...
7
8 $ ttx -l -y 4 Songti.ttc
9 Listing table info for "Songti.ttc":
10    tag      checksum      length      offset
11    ----      -
12    OS/2     0xA1E90A73      96     45395268
13    ...
```

注意 index 0 的 `os/2` 偏移 2628、index 4 的偏移 45395268——**两者完全独立**，说明 Songti.ttc 各字体的表**没有共享**（这与 msyh/cambria 不同；是否共享是各厂商的打包策略）。

3.3.4 提取每个字体的具体信息：name 表 + OS/2

`-y` 只能列出表清单。要拿到「序号 / 字重 / 名称 / 版权」这类**字体元数据**，得读 `name` 表（命名表）和 `os/2` 表。字段对应关系：

目标字段	来源
序号	TTC 内的 index（即 <code>-y</code> 值，从 0 起）
字重（数值 300/400/700/900）	<code>os/2.usweightclass</code>
英文名称 / 英文字重	<code>name</code> ：nameID=1（Family） / nameID=2（Subfamily），语言=英文
简体名称 / 简体字重	同上，语言=简体中文
繁体名称 / 繁体字重	同上，语言=繁体中文
版权年份	<code>name</code> ：nameID=0（Copyright），用正则提取年份

`name` 表里每条记录由三元组 (`platformID`, `encodingID`, `langID`) 定位语言，常见值：

语言	Windows 平台 (platformID=3)	Mac 平台 (platformID=1)
英文	(3, 1, 0x409)	(1, 0, 0)

语言	Windows 平台 (platformID=3)	Mac 平台 (platformID=1)
简体中文	(3, 1, 0x804)	(1, 25, 33)
繁体中文	(3, 1, 0x404)	(1, 2, 19)

⚠ **真实字体的 name 记录并不统一**：新字体一般只写 Windows 平台，老字体（如 STSong）常只有 Mac 平台记录。因此查询时必须 **Windows 优先、Mac 兜底**，否则会取到空值（见下文 index 4）。

3.3.5 实战：SourceHanMono.ttc 子字体名与 name 表

下面这个实战做两件事：① 列出全部子字体（第一列为解包后的 `.ttf` 文件名）；② 把单个子字体携带的**全部常用 name 记录**逐条列出来。`SourceHanMono.ttc`（思源等宽，Adobe/Google 开源）先看它装了几个子字体：

```
1 $ ttx -l SourceHanMono.ttc
2 ERROR: specify a font number between 0 and 69 (inclusive)
```

→ 共 **70 个**子字体。

① 列出全部子字体（.ttf 文件名 + Family/Subfamily）

把 TTC 解包成单独的 `.ttf` 时，文件名通常取 **PostScript 名(nameID=6)** + `.ttf`（PostScript 名是文件系统安全的唯一标识：无空格、纯 ASCII）。下面脚本遍历每个子字体，第一列给解包后的 `.ttf` 文件名，再列 Family(1)、Subfamily(2)：

```
1 from fontTools.ttLib import TTCollection
2 c = TTCollection('SourceHanMono.ttc')
3 def name(f, nid):
4     r = f['name'].getName(nid, 3, 1, 0x409) # windows 平台、英文
5     return r.toUnicode() if r else ''
6 print(f'{"filename (.ttf)":<34>{"Family(1)":<23}Subfamily(2)'}
7 print('-' * 72)
8 for f in c.fonts:
9     print(f'{name(f,6)+".ttf":<34}{name(f,1):<23}{name(f,2)}')
```

运行结果（共 70 行；为简洁仅展示首尾及代表性字重，结构相同的中间行用 `...` 省略）：

1	filename (.ttf)	Family(1)	Subfamily(2)
2	-----		
3	SourceHanMono-ExtraLight.ttf	Source Han Mono EL	Regular
4	SourceHanMono-ExtraLightIt.ttf	Source Han Mono EL	Italic
5	SourceHanMonoK-ExtraLight.ttf	Source Han Mono K EL	Regular
6	SourceHanMonoK-ExtraLightIt.ttf	Source Han Mono K EL	Italic
7	SourceHanMonoSC-ExtraLight.ttf	Source Han Mono SC EL	Regular
8	SourceHanMonoSC-ExtraLightIt.ttf	Source Han Mono SC EL	Italic
9	SourceHanMonoTC-ExtraLight.ttf	Source Han Mono TC EL	Regular
10	SourceHanMonoTC-ExtraLightIt.ttf	Source Han Mono TC EL	Italic
11	SourceHanMonoHC-ExtraLight.ttf	Source Han Mono HC EL	Regular
12	SourceHanMonoHC-ExtraLightIt.ttf	Source Han Mono HC EL	Italic
13	... (Light / Normal, 省略 index 10-29)		
14	SourceHanMono-Regular.ttf	Source Han Mono	Regular
15	SourceHanMono-RegularIt.ttf	Source Han Mono	Italic

16	...	(Regular 其余区域 + Medium, 省略 index 32-49)	
17	SourceHanMono-Bold.ttf	Source Han Mono	Bold
18	SourceHanMono-BoldIt.ttf	Source Han Mono	Bold Italic
19	...	(Bold 其余区域 + Heavy, 省略 index 52-68)	
20	SourceHanMonoHC-HeavyIt.ttf	Source Han Mono HC H	Italic

70 = 7 字重 × 5 区域 × 2 (正体/斜体) : 字重 EL/L/N/M/B/H (Regular 无缩写) ; 区域 默认 (日)/K(韩)/SC(简)/TC(繁)/HC(港)。

② 单个字体的全部 name 记录

上面只取了 Family/Subfamily。一个子字体实际携带的 name 记录远不止这些——以 index 0 (ExtraLight) 为例，把它的 name 表 (Windows 平台、英文) 逐条 dump 出来 (超长文本截断显示) :

```

1  f0 = c.fonts[0]                                # 取 index 0
2  for r in f0['name'].names:
3      if (r.platformID, r.platEncID, r.langID) == (3, 1, 0x409):
4          v = r.toUnicode()
5          print(f"{r.nameID:>2}  {v[:58]}{'...' if len(v)>58 else ''}")

```

```

1  0  © 2014–2019 Adobe (http://www.adobe.com/), with Reserved F...
2  1  Source Han Mono EL
3  2  Regular
4  3  1.002;ADBO;SourceHanMono-ExtraLight;ADOBE
5  4  Source Han Mono EL
6  5  Version 1.002;hotconv 1.0.107;makeotfexe 2.5.65593
7  6  SourceHanMono-ExtraLight
8  7  Source is a trademark of Adobe in the United States and/or...
9  8  Adobe
10 9  Ryoko NISHIZUKA 西塚涼子 (kana, bopomofo & ideographs); Paul D...
1110 Dr. Ken Lunde (project architect, glyph set definition & o...
1211 http://www.adobe.com/type/
1313 This Font Software is licensed under the SIL Open Font Lic...
1414 http://scripts.sil.org/OFL
1516 Source Han Mono
1617 EL

```

可见一个子字体携带 **16 条** name 记录 (本字体没有 nameID 12、15、18-25)。常见 nameID 含义速查:

nameID	含义	nameID	含义
0	Copyright 版权	9	Designer 设计师
1	Family 家族名	10	Description 描述
2	Subfamily 子族名	11	Vendor URL 厂商网址
3	Unique ID 唯一标识	13	License 许可证
4	Full Name 全名	14	License URL
5	Version 版本	16	Preferred Family 首选家族

nameID	含义	nameID	含义
6	PostScript 名	17	Preferred Subfamily 首选子族
7	Trademark 商标	21 / 22	WWS Family / Subfamily
8	Manufacturer 厂商	25	Variations PS Prefix (可变字体)

几个要点：

- **nameID 6 (PostScript 名)** 是文件名与唯一标识的依据，解包 `.ttf` 就用它（见 ①）。
- **"4 样式限制"技巧 (nameID 1/2 vs 16/17)**：index 0 的 Family(1) 是 `Source Han Mono EL`（字重被塞进家族名），Subfamily(2) 是 `Regular`；真正的家族/字重在 **16/17**（`Source Han Mono / EL`）。这是为兼容老软件每个家族只认 4 种样式(Regular/Italic/Bold/Bold Italic)——非标准字重重组成"假家族"；而 Regular/Bold（标准 4 样式）恰好相反：16/17 留空、真名在 1/2：

子字体	Family(1)	Subfamily(2)	PreferredFamily(16)	PreferredSubfamily(17)
index 0 ExtraLight	Source Han Mono EL	Regular	Source Han Mono	EL
index 30 Regular	Source Han Mono	Regular	(空)	(空)
index 50 Bold	Source Han Mono	Bold	(空)	(空)

思源、Noto CJK 等大家族字体普遍采用此策略。

💡 纯 ttx CLI 也能看：`ttx -y 0 -t name -o - SourceHanMono.ttc` 会把 index 0 的 name 表反编译成 XML 直接输出，可逐条核对每条 `<namerecord>`。

3.3.6 实战：Songti.ttc 完整枚举

下面的脚本对 `Songti.ttc` 的 8 个字体逐一提取上述字段（繁体缺失时回退到简体串，模拟系统字体选择器的行为）：

```
1  # -*- coding: utf-8 -*-
2  from fontTools.ttLib import TTCollection
3  import re
4
5  c = TTCollection('Songti.ttc')
6  CAND = {                                # (platformID, encodingID, langID)，按优先级排列
7      'en' : [(3,1,0x409), (1,0,0)],
8      'zhs' : [(3,1,0x804), (1,25,33)],
9      'zht' : [(3,1,0x404), (1,2,19), (1,3,21)],
10 }
11 def rec(f, nid, loc):
12     for pe in CAND[loc]:
13         r = f['name'].getName(nid, *pe)
14         if r:
```

```

15         try: return r.toUnicode()
16         except: continue
17     return ''
18 def nm(f, nid, loc):
19     v = rec(f, nid, loc)
20     if not v and loc == 'zht':          # 繁体缺失 -> 回退简体
21         v = rec(f, nid, 'zhs')
22     return v
23 def years(s):
24     return ','.join(sorted(set(re.findall(r'(?:(19|20)\d{2}(?:-\d{2,4})?', s
25 or '))), key=lambda x:x[:4]))
26 print('序号 字重 英文名称 英文字重 简体名称 简体字重 繁体名称 繁体字重 版权年份')
27 for i, f in enumerate(c.fonts):
28     w = f['OS/2'].usweightClass
29     print(i, w, nm(f,1,'en'), nm(f,2,'en'),
30           nm(f,1,'zhs'), nm(f,2,'zhs'), nm(f,1,'zht'), nm(f,2,'zht'),
31           years(nm(f,0,'en')), sep=' | ')

```

运行结果（与参考表 [YDX-Songti.pdf](#) 完全吻合）：

1	序号	字重	英文名称	英文字重	简体名称	简体字重	繁体名称	繁体字重	版权年份
2	0	900	Songti SC	Black	宋体-简	黑体	宋體-簡	黑體	1991-1998,2012
3	1	700	Songti SC	Bold	宋体-简	粗体	宋體-簡	粗体	2000-2005,2012
4	2	700	Songti TC	Bold	宋体-繁	粗体	宋體-繁	粗体	2010-2012,2013
5	3	300	Songti SC	Light	宋体-简	细体	宋體-簡	細體	1991-2001
6	4	300	STSong	Regular	华文宋体	常规体	华文宋体	標準體	2002
7	5	300	Songti TC	Light	宋体-繁	细体	宋體-繁	細體	2010-2012,2013
8	6	400	Songti SC	Regular	宋体-简	常规体	宋體-簡	標準體	2010-2012
9	7	400	Songti TC	Regular	宋体-繁	常规体	宋體-繁	標準體	2010-2012,2013

几点解读：

- **SC / TC**：Songti **SC** = Simplified Chinese（简体），Songti **TC** = Traditional Chinese（繁体）；同一字重通常成对出现（如 index 1/2 都是 Bold 700）。
- **index 4 是异类**：**STSong**（华文宋体）是更老的字体，**版权年份只有 2002**（其余多为 2010 年后），且 name 记录**只存在于 Mac 平台**——这正是上文「Mac 兜底」的来源；它也没有专门的繁体记录，故繁体列回退显示简体串「华文宋体」。
- **测试文本列**：参考表里的「然而永远！分的喆冇砵？」是**固定的样张字符串**（喆 / 冇 / 砵 是生僻字，用来抽测字形覆盖），**并非字体元数据**，需自行定义与渲染；原参考图用思源黑体补齐缺字。
- **字重数值**：300=Light、400=Regular、700=Bold、900=Black，来自 `os/2.usweightClass`（OpenType 标准 weight 值）。

💡 **纯 ttx CLI 也能看 name 表**：`ttx -y 4 -t name -o - Songti.ttc` 会把 index 4 的 name 表反编译成 XML 打到标准输出，可直接看到每条 `<namerecord nameID="..." platformID="..." encodingID="..." langID="...">` 的对应关系，便于人工核对。

3.3.7 什么是 Super TTC (Super OTC)

前面 3.3.5 的 `SourceHanMono.ttc` 一个文件装了 **70 个子字体** 却体积可控——这背后是 **Super TTC** (Adobe 称 **Super OTC**)：**把整个家族 (全部字重 × 全部语言/区域)** 塞进一个集合文件，**靠最大化 sfnt 表共享** 来压缩体积。

普通 TTC 和 Super TTC **底层机制相同** (都是 sfnt 表按偏移共享)，区别只在共享的彻底程度。对比：

维度	普通 TTC/OTC	Super TTC/OTC
打包范围	任意若干字体 (常一个字重或几个)	整个家族：全部字重 × 全部语言/区域
实例数	少 (msyh 2、cambria 2、Songti 8)	多 (几十个：Source Han Sans 45、SourceHanMono 70)
表共享	字节相同的表按偏移共享 (msyh 25 张表共享 19 张)	同机制，但最大化去重 (见下表)
体积收益	中等	最大 (比独立 OTC 省 13 MB+)

Super TTC 在打包阶段就按字重/语言维度做了最大化去重——最大的几张表这样分摊：

表	共享维度	唯一份数
<code>CFF</code> / <code>hmtx</code> / <code>vmtx</code>	按字重	每个字重一份
<code>GSUB</code>	按语言/区域	每个语言一份
<code>cmap</code>	按 语言 × 比例/半宽	每组合一份

以 Adobe 官方 Source Han Sans 的 **Super OTC** 为例：7 个字重 × 5 个语言 (+ Regular/Bold 的半宽变体) = **45 个字体实例、458,745 个字形**，却只有 **7 个 CFF、5 个 GSUB、10 个 cmap**——相比 7 个独立的按字重 OTC **省下 13 MB+**。

我们的 `SourceHanMono.ttc` 就是这种结构。用 `ttx -l -y` 看不同 index 的 `CFF` 表偏移，能直接看到“同一字重共享一个 CFF”：

1	\$ ttx -l -y 0 SourceHanMono.ttc	# ExtraLight, default, Regular
2	CFF 0x911751C3 15010995 20732	
3	\$ ttx -l -y 4 SourceHanMono.ttc	# ExtraLight, SC, Regular
4	CFF 0x911751C3 15010995 20732	← 与 index 0 同一份 CFF (校验和、偏移全同)
5	\$ ttx -l -y 30 SourceHanMono.ttc	# Regular, default
6	CFF 0x0171E9C8 16253281 47377044	← 另一份 (Regular 字重)
7	\$ ttx -l -y 50 SourceHanMono.ttc	# Bold, default
8	CFF 0x67DEB2E5 16858190 79990176	← 又一份 (Bold 字重)

index 0 与 4 (同为 ExtraLight、不同区域) 的 CFF 偏移都是 **20732**、校验和相同——它们**共用同一份 CFF**；换字重 (Regular→47377044、Bold→79990176) 才是另一份。所以 70 个子字体实际只有 **7 个 CFF** (每个字重一份，由该字重的 10 个“区域 × 正/斜体”变体共享)，这就是它能塞下 70 个字体的原因。


```
1 0 Sarasa-Gothic-CL-ExtraLight
2 1 Sarasa-Gothic-SC-ExtraLight
3 2 Sarasa-Gothic-TC-ExtraLight
4 3 Sarasa-Gothic-HC-ExtraLight
5 4 Sarasa-Gothic-J-ExtraLight
6 5 Sarasa-Gothic-K-ExtraLight
7 6 Sarasa-UI-CL-ExtraLight
8 7 Sarasa-UI-SC-ExtraLight
9 8 Sarasa-UI-TC-ExtraLight
10 9 Sarasa-UI-HC-ExtraLight
11 10 Sarasa-UI-J-ExtraLight
12 11 Sarasa-UI-K-ExtraLight
```

前 12 个都是 ExtraLight 字重、两个家族 (Gothic / UI) × 六个区域 (CL/SC/TC/HC/J/K) , 可见 480 = 若干字重 × 家族 × 区域 的组合。

⚠️ 坑二：拆解后体积暴涨（与普通 TTC 的关键差异）

Super TTC 靠**共享 glyph** 压缩体积（同一字重的全部字体共用一份 ~44 MB 的 **glyph**）。一旦拆成独立 **.ttf**，每个文件都要自带一份 **glyph**，共享红利瞬间消失。以较小的单字重 **Sarasa-SemiBold.ttc** 实测（拆出 index 0）：

```
1 $ ls -l Sarasa-SemiBold.ttc
2 79 MB                               ← 原始 TTC（48 字体，glyph 共享 1 份）
3 $ ls -l Sarasa-Gothic-CL-SemiBold.ttf
4 43 MB                               ← 单个就 43 MB（自带完整 glyph）
```

	原始 TTC	拆成独立 .ttf
前 12 个	(共享中)	12 × 43 MB ≈ 500 MB
全部 48 个 (单字重)	79 MB	48 × 43 MB ≈ 2 GB (26×)
全部 480 个 (SuperTTC)	793 MB	≈ 20 GB (不现实)

所以——**只拆你真正需要的那个 index**，绝不要整包拆。

普通 TTC vs Super TTC 拆解对比：

维度	普通 TTC (Sarasa-SemiBold)	Super TTC (Sarasa-SuperTTC)
子字体数	48	480
TTCollection() 整包载入	可行 (79 MB)	OOM (793 MB, 必须逐个开)
单字体拆解 API	<code>TTFont(f, fontNumber=N).save()</code>	相同
整包拆解后体积	~2 GB (26×)	~20 GB (不现实)
建议	按需拆	只拆需要的 index

💡 **挑选 index**: 不知道某个 index 是哪个字体时, 先开它读 PostScript 名 (nameID=6) 即可——`TTFont(f, fontNumber=N)['name'].getName(6, 3, 1, 0x409)`, 无需保存文件。这与 3.3.5 / 3.3.6 的"Name 表"手法一脉相承。

3.4 TrueType vs OpenType/CFF: 两种轮廓格式

这是理解字体的**核心区分点**, 也直接决定 ttx 输出的差异:

维度	TrueType (.ttf)	OpenType/CFF (.otf)
轮廓表	glyph + loca	CFF
曲线类型	二次贝塞尔 (控制点 on / off)	三次贝塞尔
hinting	TrueType 指令 (fpgm / prep / cvt)	CFF 提示 (Private dict 里的 BlueValues 等)
maxp 表	32 字节, 含 15 个限制字段	仅 6 字节 (版本 + numGlyphs)
后缀	.ttf	.otf (但 .otf 也能装 glyph)

注意: **文件后缀不绝对等于轮廓类型**。 .otf 既可以装 CFF 也可以装 glyph, 判定依据是表里有没有 CFF 表还是 glyph 表。

第四部分 Dump 操作实战

4.1 反编译: 字体 → TTX

```
1 $ ttx -v -d ttx_out times.ttf
2 Dumping "times.ttf" to "ttx_out\times.ttx"...
3 Dumping 'GlyphOrder' table...
4 Reading 'post' table from disk
5 Decompile 'post' table
6 Reading 'maxp' table from disk
7 Decompile 'maxp' table
8 ... (略, 逐表 Reading / Decompile / Dumping) ...
9 Reading 'DSIG' table from disk
10 Decompile 'DSIG' table
11 Dumping 'DSIG' table...
12 Done dumping TTX in 6.727 seconds
```

产物 `times.ttx` 约 **22.5 MB** (原始字体 1.2 MB, XML 化后膨胀约 18 倍)。 `-v` 让每个表的"读盘→反编译→写出"步骤都可见; 不带 `-v` 则只打印一行汇总。

4.2 编译: TTX → 字体 (无损往返)

```
1 $ ttx -d ttx_out ttx_out/times.ttx
2 Compiling "ttx_out/times.ttx" to "ttx_out\times.ttf"...
3 Parsing 'GlyphOrder' table...
4 Parsing 'head' table...
5 ... (逐表 Parsing) ...
6 Parsing 'DSIG' table...
```

体积对比:

```
1 原 times.ttf      : 1201620 字节
2 重编译 times.ttf  : 1196136 字节
```

说明: 往返后体积略有差异 (小约 0.5%) 是正常的——ttx 在编译时会重新计算部分字段 (如校验和、某些 bbox), 二进制内部排列也可能与原始工具不同, 但字形数据与字体功能等价。这就是“无损往返”的含义: 视觉与功能一致, 而非逐字节相同。

4.3 输出控制 -d / -o / -f

-d 指定目录 (必须事先存在)

```
1 $ ttx -s -q -d ttx_out/split_t times.ttf
2 ERROR: The -d option value must be an existing directory
```

→ 需先 `mkdir -p ttx_out/split_t` 再运行。

-o - 输出到标准输出

```
1 $ ttx -t head -o - times.ttf
2 Dumping "times.ttf" to "<stdout>"...
3 <?xml version="1.0" encoding="UTF-8"?>
4 <ttFont sfntVersion="\x00\x01\x00\x00" ttLibVersion="4.61">
5   <head>
6     ...
7   </head>
8 </ttFont>
```

适合管道处理 (如 `ttx -t head -o - font.ttf | grep modified`)。

-f 覆盖 vs 默认唯一命名

默认永不覆盖, 自动追加 #N:

```
1 第1次 ttx times.ttf -> ['times.ttf']
2 第2次 ttx times.ttf (默认不覆盖) -> ['times#1.ttf', 'times.ttf']
3 第3次 ttx -f times.ttf (覆盖) -> ['times#1.ttf', 'times.ttf'] # 直接覆盖 times.ttf, 不产生 #2
```

⚠ 重要陷阱: 当 `-o` 和 `-d` 同时出现时, `-o` 生效、`-d` 被忽略, 文件按 `-o` 的值 (相对当前工作目录) 写出, 而非落到 `-d` 目录里。要写到指定目录, 应写 `-o 目录/文件.ttf`。

4.4 详细与静默 `-v` / `-q`

同样只导出 `head` 表：

```
1 --- verbose (-v) ---
2 Dumping "times.ttf" to "<stdout>"...
3 Reading 'head' table from disk
4 Decompiling 'head' table
5 Dumping 'head' table...
6 Done dumping TTX in 0.006 seconds
7
8 --- quiet (-q) ---
9 （无任何进度输出，只有 XML 数据本身）
```

4.5 选择性导出 `-t` / `-x`

`-t` 只导出指定表（可多个）

```
1 $ ttx -q -t head -t name -t maxp -d ttx_out times.ttf
```

导出方式	文件大小
完整 times.ttf（全部表）	22 549 613 字节（22.5 MB）
<code>-t head,name,maxp</code> （仅 3 表）	14 515 字节（14 KB）

`-x` 排除指定表（可多个）

```
1 $ ttx -q -x glyf -x hdmx -x GPOS -d ttx_out times.ttf
```

排除 3 个大表后 → **2 383 480 字节**（2.38 MB）。`-t` 与 `-x` **互斥**，不能同一条命令里混用。

实用技巧：只想看字体的元数据而不关心庞大轮廓时，`-x glyf` 能把 22 MB 砍到 2 MB，极大便于阅读和 diff。

4.6 拆分 `-s` / `-g`

`-s` 按表拆分

```
1 $ ttx -q -s -d ttx_out/split_t times.ttf
```

生成 **27 个文件**（每个表一个 `.ttx`），另有一个主文件用 `src=` 属性引用它们：

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <ttFont sfntVersion="\x00\x01\x00\x00" ttLibVersion="4.61">
3   <GlyphOrder src="times.GlyphOrder.ttx"/>
4   <head src="times._h_e_a_d.ttx"/>
5   <hhea src="times._h_h_e_a.ttx"/>
6   <maxp src="times._m_a_x_p.ttx"/>
7   <OS_2 src="times.O_S_2f_2.ttx"/>
8   <hmtx src="times._h_m_t_x.ttx"/>
9   ...
10  <glyph src="times._g_l_y_f.ttx"/>
11  ...
12 </ttFont>

```

注意表标签里的空格用下划线转义（如 `cvt` → `_c_v_t`）。这个主文件可以直接喂回 `ttx` 编译，只要各分表文件在同目录。

-g 按字形拆分 glyf

```
1 $ ttx -q -g -d ttx_out/split_g times.ttf
```

生成 **4750 个文件**，其中 **4724 个字形文件**，命名形如：

```

1 times._g_l_y_f..notdef.ttx
2 times._g_l_y_f.A_.ttx
3 times._g_l_y_f.A_E_.ttx
4 times._g_l_y_f.A_E_acute.ttx
5 ...

```

`-g` 隐含 `-s`（无需再加 `-s`）。适合在 Git 里单独追踪每个字形的修改，或精确定位某字形问题。

4.7 TTC 字体集合 -y

`.ttc/.otc` 含多个字体，必须用 `-y` 选编号（从 0 起）：

```

1 $ ttx -l cambria.ttc
2 ERROR: specify a font number between 0 and 1 (inclusive)
3
4 $ ttx -y 1 -t MATH -o - cambria.ttc    # 导出编号1(数学字体)的 MATH 表

```

4.8 -i 不反汇编 TrueType 指令

TrueType 的 hinting 是一套栈式字节码指令。默认 `ttx` 把它**反汇编成可读汇编**；加 `-i` 则保留为**十六进制**，更快更小。

默认（反汇编）：

```

1 <fpgm>
2   <assembly>
3     NPUSHB[ ] /* 87 values pushed */
4     133 116 115 114 113 112 ... (略)
5     FDEF[ ] /* FunctionDefinition */
6     RCVT[ ] /* ReadCVT */
7     SWAP[ ] /* SwapTopStack */
8     GC[0] /* GetCoordOnPVector */
9     ADD[ ] /* Add */
10    DUP[ ] /* DuplicateTopStack */
11    ...

```

加 `-i` (十六进制) :

```

1 <fpgm>
2   <bytecode>
3     40578574 73727170 6f6e6d6c 6b6a6968
4     67666562 5d55544f 4e403f3e 3d3c3b3a
5     39383736 35343332 31302f2e 2d2c2b2a
6     ...

```

区别：默认用 `<assembly>` 元素 (含助记符 + 注释, 可读但占空间) ; `-i` 用 `<bytecode>` 元素 (紧凑 hex) 。两者编译回字体后功能等价。

4.9 -z 位图导出格式

针对 `EBDT/CBDT` (嵌入式点阵位图) 表, `-z` 控制位图数据的 XML 表达。以 Cambria 字形 A (8×7 点阵) 为例:

`-z raw` (整块十六进制) :

```

1 <rawimagedata>
2   10102828 287c44ee
3 </rawimagedata>

```

`-z row` (逐行十六进制, 每行一个字节) :

```

1 <rowimagedata bitDepth="1" height="8" width="7">
2   <row value="10"/>
3   <row value="10"/>
4   <row value="28"/>
5   <row value="28"/>
6   <row value="28"/>
7   <row value="7c"/>
8   <row value="44"/>
9   <row value="ee"/>
10 </rowimagedata>

```

`-z bitwise` (ASCII 点阵艺术, @=黑 . =白) :

```

1 <bitwiseimagedata bitDepth="1" height="8" width="7">
2   <row value="...@..." />
3   <row value="...@..." />
4   <row value="..@.@" />
5   <row value="..@.@" />
6   <row value="..@.@" />
7   <row value="..@.@" />
8   <row value="..@.@" />
9   <row value="..@.@" />
10 </bitwiseimagedata>

```

`bitwise` 格式下能肉眼直接看出字母 A 的形状，是检视点阵字形最直观的方式。另有 `-z` `extfile` 把位图导出为外部文件、主 XML 里放引用。CBDT（彩色位图）只支持 `raw` 和 `extfile`。默认格式是 `raw`。

4.10 --newline 行尾控制

控制写出的 XML 文件用哪种换行符：

```

1 $ ttx -q --newline LF -t head -o newline_lf.ttx times.ttf
2 $ ttx -q --newline CRLF -t head -o newline_crlf.ttx times.ttf

```

校验（统计字节数据）：

```

1 LF 模式文件：CRLF = 0 次，纯 LF = 25 次
2 CRLF 模式文件：纯 LF(无 CR) = 0 次

```

跨平台协作（尤其在 Windows 上生成、Linux 上用 Git 管理）时，统一用 `--newline LF` 可避免无意义的换行符 diff。

第五部分 Compile 操作实战

5.1 --flavor woff2 输出 WOFF2

把 TTX 编译成 WOFF2（需安装 brotli）：

```

1 $ ttx --flavor woff2 -o ttx_out/times.woff2 ttx_out/times.ttf

```

体积对比：

```

1 原 times.ttf    : 1201620 字节
2 WOFF2          :   506396 字节
3 压缩率          : 42.1%

```

WOFF2 用 Brotli 压缩，通常能把字体压到原大小的 30%–50%，是 Web 字体的主流格式。`--flavor woff` 则生成 WOFF（Zlib 压缩，可配 `--with-zopfli` 提高压缩率）。

5.2 --recalc-timestamp 时间戳控制

字体的 `modified` 时间戳默认用 TTX 文件的修改时间。两种显式控制：


```
1 原始 times.ttf head.modified:
2    <modified value="Fri Jun 17 17:02:27 2022"/>
3
4 编译 --no-recalc-timestamp (保留原值):
5    <modified value="Fri Jun 17 17:02:27 2022"/>
6
7 编译 --recalc-timestamp (设为当前时间):
8    <modified value="Thu Jul 23 05:58:34 2026"/>
```

发布字体时通常希望时间戳反映本次构建时间 → 用 `--recalc-timestamp`。

5.3 -m 合并 (只改几张表的工作流)

`-m` 把一个 TTX 文件合并进一个已有字体：以原字体为底，TTX 里出现的表覆盖原表，其余表保留。例：`times#1.ttx` 只含 `head + name + maxp`，把它合并进原字体：

```
1 $ ttx -m times.ttf -o ttx_out/merged.ttf ttx_out/times#1.ttx
```

```
1 合并产物 merged.ttf : 1201688 字节 (基本=原字体, 仅 3 张表被 TTX 覆盖)
2 原 times.ttf          : 1201620 字节
```

这是“只改某张表再回编”的标准流程：导出单表 → 改 XML → 用 `-m` 合并回原字体，无需处理全部表。限制：最多只能指定一个 TTX 文件。

5.4 其余编译选项

选项	用途
<code>-b</code>	不重算字形包围盒，用 TTX 里的原值（默认会重算）
<code>--with-zopfli</code>	生成 WOFF 时用 Zopfli 压缩（更小更慢，需装 zopfli）
<code>--optimize-font-speed</code>	编译时优化渲染速度优先于体积（产物更大，HarfBuzz 等渲染更快），主要影响 <code>glyf</code> / <code>gvar</code> / <code>VARC</code>

第六部分 TTX (XML) 格式逐表解读

每个 TTX 文件的根元素是 `<ttFont>`，`sfntVersion` 标识类型：`\x00\x01\x00\x00` = TrueType，`OTTO` = OpenType/CFF。

6.1 head 字体头

```
1 <head>
2   <!-- Most of this table will be recalculated by the compiler -->
3   <tableVersion value="1.0"/>
4   <fontRevision value="7.05"/>           <!-- 字体修订版本号 -->
5   <checksumAdjustment value="0x328bad8a"/> <!-- 校验和调整（编译时重算）-->
6   <magicNumber value="0x5f0f3cf5"/>      <!-- 魔数，恒为 0x5F0F3CF5 -->
7   <flags value="00001000 00011001"/>     <!-- 全局标志位 -->
8   <unitsPerEm value="2048"/>            <!-- em 单位（坐标系的分辨率）-->
```

```

9      <created value="Mon Aug 6 13:14:42 1990"/>
10     <modified value="Fri Jun 17 17:02:27 2022"/>
11     <xMin value="-1164"/> <yMin value="-628"/>
12     <xMax value="4190"/> <yMax value="2129"/> <!-- 所有字形的包围盒 -->
13     <macStyle value="00000000 00000000"/> <!-- 粗体/斜体等风格位 -->
14     <lowestRecPPEM value="9"/> <!-- 最低推荐像素大小 -->
15     <fontDirectionHint value="1"/>
16     <indexToLocFormat value="1"/> <!-- loca 表偏移格式
      (0=short,1=long) -->
17     <glyphDataFormat value="0"/>
18 </head>

```

`unitsPerEm` 是关键：所有字形坐标都以它为分母，Times New Roman 用 2048，思源黑体用 1000。

6.2 name 命名表

每个 `namerecord` 用 `nameID` + `platformID`/`platEncID`/`langID` 三元组定位一条字符串：

```

1 <name>
2   <namerecord nameID="0" platformID="0" platEncID="3" langID="0x0">
3     © 2020 The Monotype Corporation. All Rights Reserved. ...
4   </namerecord>
5   <namerecord nameID="1" platformID="0" platEncID="3" langID="0x0">
6     Times New Roman <!-- nameID=1 字体家族名 -->
7   </namerecord>
8   <namerecord nameID="2" platformID="0" platEncID="3" langID="0x0">
9     Regular <!-- nameID=2 子家族名(样式) -->
10  </namerecord>
11  <namerecord nameID="4" platformID="0" platEncID="3" langID="0x0">
12    Times New Roman <!-- nameID=4 完整字体名 -->
13  </namerecord>
14  <namerecord nameID="5" platformID="0" platEncID="3" langID="0x0">
15    Version 7.05 <!-- nameID=5 版本字符串 -->
16  </namerecord>
17  <namerecord nameID="6" platformID="0" platEncID="3" langID="0x0">
18    TimesNewRomanPSMT <!-- nameID=6 PostScript 名 -->
19  </namerecord>
20  ...

```

常用 `nameID`：0=版权，1=家族名，2=子家族名，3=唯一ID，4=完整名，5=版本，6=PostScript 名，7=商标，8=厂商，9=设计师，13=许可证。

6.3 hhea / hmtx 水平度量

`hhea` 给出整体水平排版参数：

```

1 <hhea>
2   <tableVersion value="0x00010000"/>
3   <ascent value="1825"/>      <descent value="-443"/>  <!-- 上升/下降 -->
4   <lineGap value="87"/>
5   <advanceWidthMax value="4096"/>  <!-- 最大前进宽度 -->
6   <numberOfHMetrics value="4729"/>  <!-- 字形数 -->
7   ...
8 </hhea>

```

`hmtx` 给每个字形的**前进宽度**(width)和**左侧间距**(lsb):

```

1 <hmtx>
2   <mtx name=".notdef" width="1593" lsb="284"/>
3   <mtx name="A" width="1479" lsb="16"/>
4   <mtx name="AE" width="1821" lsb="-24"/>
5   ...
6 </hmtx>

```

6.4 maxp: TrueType vs CFF 对比

这是两种轮廓格式差异最直观的体现:

TrueType (times.ttf, 32 字节, 15 个限制字段) :

```

1 <maxp>
2   <tableVersion value="0x10000"/>
3   <numGlyphs value="4729"/>
4   <maxPoints value="693"/>      <!-- 单字形最多点数 -->
5   <maxContours value="60"/>    <!-- 最多轮廓数 -->
6   <maxCompositePoints value="209"/>
7   <maxCompositeContours value="7"/>
8   <maxZones value="2"/>      <!-- hinting 区域数 -->
9   <maxTwilightPoints value="16"/>
10  <maxStorage value="64"/>
11  <maxFunctionDefs value="134"/>  <!-- 最多函数定义(TrueType指令) -->
12  <maxStackElements value="1513"/>
13  <maxSizeOfInstructions value="3987"/>
14  ...
15 </maxp>

```

CFF (思源黑体, 仅 6 字节) :

```

1 <maxp>
2   <tableVersion value="0x5000"/>
3   <numGlyphs value="65535"/>
4 </maxp>

```

CFF 字体不需要 TrueType 那套指令内存限制, 所以 `maxp` 退化到只剩版本和字形数。

`tableVersion` 也不同: `0x10000` = TrueType, `0x5000` = CFF。

6.5 OS/2

```

1 <OS_2>
2   <version value="3"/>
3   <usweightClass value="400"/>      <!-- 字重: 100-900, 400=Regular -->
4   <uswidthClass value="5"/>        <!-- 字宽: 1-9, 5=Medium(normal) -->
5   <panose> ... </panose>           <!-- PANOSE 分类(10维视觉特征) -->
6   <achVendID value="TMC "/>        <!-- 厂商标识 -->
7   <STypoAscender value="1420"/>
8   <STypoDescender value="-442"/>
9   <STypoLineGap value="307"/>
10  <usWinAscent value="1825"/>
11  <usWinDescent value="443"/>
12  <sxHeight value="916"/>          <!-- x 字高 -->
13  <scapHeight value="1356"/>       <!-- 大写字高 -->
14  ...
15 </OS_2>

```

`usweightClass` 决定字重 (100 细 → 900 黑) , `PANOSE` 是 10 维字体视觉分类。

6.6 post

```

1 <post>
2   <formatType value="3.0"/>      <!-- 版本3=不含字形名(省空间) -->
3   <italicAngle value="0.0"/>
4   <underlinePosition value="-223"/>
5   <underlineThickness value="100"/>
6   <isFixedPitch value="0"/>     <!-- 是否等宽 -->
7   ...
8 </post>

```

`formatType 2.0` 含完整字形名表, `3.0` 省略 (现代字体常用以减小体积) 。

6.7 cmap 字符映射

把 Unicode 码点映射到字形名。CJK 字体的 cmap 极其丰富：

```

1 <cmap>
2   <tableVersion version="0"/>
3   <cmap_format_4 platformID="0" platEncID="3" language="0">
4     ...
5     <map code="0x20" name="space"/><!-- SPACE -->
6     <map code="0x21" name="exclam"/><!-- EXCLAMATION MARK -->
7     <map code="0x22" name="quotedbl"/><!-- QUOTATION MARK -->
8     <map code="0x23" name="numbersign"/><!-- NUMBER SIGN -->
9     <map code="0x24" name="dollar"/><!-- DOLLAR SIGN -->
10    ...

```

注释里的字符名 (`<!-- SPACE -->`) 来自 Unicode 数据库, 可用 `--unicodedata` 自定义。一个字体通常含多个子表 (`cmap_format_4` UCS-2、`cmap_format_12` 全 Unicode、`cmap_format_14` 异体序列等) , 覆盖不同平台/编码。

6.8 glyf TrueType 轮廓

每个字形是一个 `<TTGlyph>`，含若干 `<contour>`（轮廓），轮廓由点组成，`on` 控制点是顶点、`off` 是二次贝塞尔控制点。字形 `I`：

```
1 <TTGlyph name="I" xMin="51" yMin="0" xMax="632" yMax="1356">
2   <contour>
3     <pt x="632" y="37" on="1"/>      <!-- on=轮廓顶点 -->
4     <pt x="632" y="0" on="1"/>
5     <pt x="51" y="0" on="1"/>
6     <pt x="51" y="37" on="1"/>
7     <pt x="99" y="37" on="1"/>
8     <pt x="183" y="37" on="0"/>      <!-- off=二次贝塞尔控制点 -->
9     <pt x="221" y="86" on="1"/>
10    <pt x="245" y="118" on="0"/>
11    <pt x="245" y="240" on="1"/>
12    ...
```

复合字形（如带重音的字母）用 `<component>` 引用其他字形。

6.9 CFF PostScript 轮廓 (CID-keyed)

思源黑体的 CFF 表展示了 **CID-keyed** 结构（CJK 字体典型）：

```
1 <CFF>
2   <major value="1"/> <minor value="0"/>
3   <CFFFont name="SourceHanSansSC-Regular">
4     <ROS Registry="Adobe" Order="Identity" Supplement="0"/> <!-- CID
Registry/Ordering -->
5     <FullName value="Source Han Sans Simplified Chinese Regular"/>
6     <FontMatrix value="0.001 0 0 0.001 0 0"/> <!-- 1/1000 缩放 -->
7     <FontBBox value="-1002 -1048 2928 1808"/>
8     <CIDFontVersion value="2.00500011"/>
9     <CIDCount value="65535"/> <!-- CID 总数 -->
10    <FDSelect format="3"/>
11    <FDArray>
12      <FontDict index="0">
13        <FontName value="SourceHanSansSC-Regular-Alphabetic"/>
14        <Private>
15          <BlueValues value="-13 0 544 557 735 747"/> <!-- CFF hinting 对齐区
-->
16          <OtherBlues value="-250 -229"/>
17          <StdHW value="78"/> <StdVW value="85"/> <!-- 标准笔画宽 -->
18          <StemSnapH value="78 111"/> <StemSnapV value="85 95"/>
19          ...
```

关键概念：

- **ROS** (Registry/Ordering/Supplement) 声明这是 CID 字体，用 Adobe-Identity-0 排序。
- **CIDCount** 65535 = CID 编号空间上限。
- **FDArray / FDSelect**：把字形分组到多个 FontDict（不同字符集用不同 hinting 参数），是 CJK CID 字体管理海量字形的方式。

- CFF 的 hinting 在 `Private` dict 里 (`BlueValues` 对齐区、`StdHW` 笔画宽)，与 TrueType 的指令式 hinting 完全不同。

6.10 MATH 数学排版表

Cambria Math 独有，定义数学排版常量与字形变体：

```
1 <MATH>
2   <Version value="0x00010000"/>
3   <MathConstants>
4     <ScriptPercentScaleDown value="73"/>          <!-- 下标缩放到 73% -->
5     <ScriptScriptPercentScaleDown value="60"/>    <!-- 二级下标缩放到 60% -->
6     <DelimitedSubFormulaMinHeight value="3000"/>
7     <DisplayOperatorMinHeight value="2500"/>      <!-- 显示模式算符最小高度 -->
8     <MathLeading><Value value="300"/></MathLeading>
9     <AxisHeight>                                   <!-- 数学轴高度(分数线位置) -->
10  >
11    <Value value="585"/>
12    <DeviceTable> ... </DeviceTable>              <!-- 按字号的微调表 -->
13  </AxisHeight>
14  <AccentBaseHeight><Value value="976"/></AccentBaseHeight>
15  <SubscriptShiftDown>                             <!-- 下标下移量 -->
16    <Value value="418"/>
17    <DeviceTable> ... </DeviceTable>
18  </SubscriptShiftDown>
19  ...
```

MATH 表还含 `MathGlyphInfo` (拉伸字形、斜体修正) 和 `MathVariants` (巨型括号、根号等的变体组装)，是 Word 公式编辑器、LaTeX 渲染器做高质量数学排版的基础。

6.11 GSUB / GPOS OpenType 布局

GSUB (字形替换) 定义 ScriptList/FeatureList/LookupList 三级结构：

```
1 <GSUB>
2   <Version value="0x00010000"/>
3   <ScriptList>
4     <!-- ScriptCount=5 -->
5     <ScriptRecord index="0">
6       <ScriptTag value="arab"/>                  <!-- 阿拉伯文脚本 -->
7       <Script>
8         <DefaultLangSys>
9           <ReqFeatureIndex value="65535"/>
10          <!-- FeatureCount=11 -->
11          <FeatureIndex index="0" value="5"/>
12          <FeatureIndex index="1" value="10"/>
13          ...
14        </DefaultLangSys>
15        <LangSysRecord index="0">
16          <LangSysTag value="FAR"/>                <!-- 波斯语 -->
17          <LangSys> ... </LangSys>
```

结构：脚本(Script, 如 `latn` / `arab` / `hani`) → 语言(LangSys, 如 `ENG`) → 特性(Feature, 如 `liga` 连字 / `kern` 字距) → 查找(Lookup, 具体替换规则)。

- **GSUB** 做替换：连字(f+i→fi)、风格替换、小型大写等。
- **GPOS** 做定位：字距调整(kern)、重音标注定位(mark / mkmk)、连音符等。

6.11.1 GPOS 字距调整实例

GPOS 与 GSUB 共用同一套"脚本→语言→特性→查找"骨架，区别只在 **Lookup 的类型**——GPOS 的查找描述"如何移动/对齐字形"。Times New Roman 的 GPOS 含 4 类特性：

特性	含义
kern	成对字距调整（拉近/推远）
mark	重音附着到基础字形（如 á 中 ´ 贴到 a）
mkmk	标注附着到标注（叠加重音）
csp	大写字母间距

kern 特性在 FeatureList 中指向若干 Lookup：

```
1 <FeatureRecord index="5">
2   <FeatureTag value="kern"/>
3   <Feature>
4     <!-- LookupCount=10 -->
5     <LookupListIndex index="0" value="21"/>
6     ...
7   </Feature>
8 </FeatureRecord>
```

最核心的是这些 Lookup 里的 **PairPos（查找类型 2，成对字距）**。Times 用"基于类别(class)"的 Format 2：把"第一个字形"和"第二个字形"各自分组，再用一张行×列矩阵给出每组组合的间距，远比逐对列举紧凑：

```
1 <Lookup index="0">
2   <LookupType value="9"/> <!-- 9=Extension: 大表用 32 位扩展指针 -->
3   <LookupFlag value="8"/><!-- ignoreMarks -->
4   <ExtensionPos index="0" Format="1">
5     <ExtensionLookupType value="2"/> <!-- 实际类型: PairPos -->
6     <PairPos Format="2">
7       <Coverage> <!-- 参与"第一字形"的字形 -->
8         <Glyph value="A"/> ... <Glyph value="T"/> <Glyph value="V"/> ...
9       </Coverage>
10      <ValueFormat1 value="4"/> <!-- 4=只存 XAdvance（水平推进量） -->
11      <ClassDef1> <!-- 第一字形分组 -->
12        <ClassDef glyph="A" class="3"/> ...
13      </ClassDef1>
14      <ClassDef2> <!-- 第二字形分组 -->
15        ... <ClassDef glyph="T" class="11"/> <ClassDef glyph="V"
16        class="12"/> ...
17      </ClassDef2>
18      <!-- 行=第一字形类，列=第二字形类，交叉处即字距 -->
19      <Class1Record index="3"> <!-- 第 3 行 = A -->
```

```

19         <Class2Record index="11"><Value1 XAdvance="-227"/></Class2Record>
    <!-- A+T -->
20         <Class2Record index="12"><Value1 XAdvance="-264"/></Class2Record>
    <!-- A+V -->
21         ...
22     </Class1Record>
23 </PairPos>
24 </ExtensionPos>
25 </Lookup>

```

读法: `<Value1 XAdvance="-264"/>` 表示当 A 后接 V 时, 把 A 的水平推进量减少 264 (unitsPerEm=2048, 约缩进 13%), 让 AV 这类斜线形组合收紧、视觉间距均匀——这就是字距调整 (kerning) 的本质。负值拉近、正值推开。

其他 GPOS 查找类型: 类型 4 `MarkBasePos` (重音附着, `mark` 特性)、类型 6 `MarkMarkPos` (标注叠标, `mkmk`)、类型 1 `SinglePos` (单字形单独定位)。`ValueFormat` 是位掩码, 决定 `ValueRecord` 存哪些量: 4=XAdvance、8=YAdvance、1=XPlacement 等。

6.12 vhea / vmtx 纵向排版 (CJK)

CJK 字体需要纵向 (竖排) 度量:

```

1 <vhea>
2   <tableVersion value="0x00010000"/>
3   <ascent value="1740"/> <descent value="-308"/>
4   <advanceHeightMax value="2048"/> <!-- 最大前进高度(竖排行进方向) -->
5   <numberOfVMetrics value="1"/>
6   ...
7 </vhea>

```

```

1 <vmtx>
2   <mtx name=".notdef" height="2048" tsb="269"/> <!-- height=前进高度, tsb=顶部
   间距 -->
3   <mtx name="A" height="2048" tsb="249"/>
4   ...
5 </vmtx>

```

6.13 VORG / BASE

VORG (CFF 字体纵向原点) 记录字形在竖排时的垂直原点:

```

1 <VORG>
2   <defaultVertOriginY value="880"/> <!-- 默认纵向原点 Y -->
3   <VOriginRecord>
4     <glyphName value="cid00736"/> <!-- 需要单独调整的字形(CID名) -->
5     <vorigin value="867"/>
6   </VOriginRecord>
7   ...
8 </VORG>

```

BASE 表定义不同脚本的基线:


```

1 <BASE>
2   <HorizAxis>
3     <BaseTagList>
4       <BaselineTag index="0" value="icfb"/> <!-- 表意字下基线 -->
5       <BaselineTag index="1" value="icft"/> <!-- 表意字上基线 -->
6       <BaselineTag index="2" value="ideo"/> <!-- 表意字基线 -->
7       <BaselineTag index="3" value="romn"/> <!-- 罗马字基线 -->
8     </BaseTagList>
9     ...

```

`icfb/icft/ideo` 是 CJK 表意字专用基线，让混合排版时不同脚本对齐正确。

6.14 hinting: fpgm / prep / cvt

TrueType 字体的 hinting 由三张表配合（见 4.8 节看 `fpgm` 内容）：

- `fpgm` (font program)：字体级函数定义，全局只执行一次，定义供字形调用的函数。
- `prep` (pre-program)：每次字号/分辨率变化时执行，设置控制值和状态。
- `cvt` (control value table)：一组预定义数值，hinting 程序用它对齐笔画到像素网格：

```

1 <cvt>
2   <cv index="0" value="1422"/> <!-- 常与大写高度相关 -->
3   <cv index="2" value="1356"/> <!-- cap height -->
4   <cv index="6" value="916"/> <!-- x-height -->
5   <cv index="14" value="-438"/> <!-- descender 相关 -->
6   ...
7 </cvt>

```

`cvt` 里的值（如 1356=cap height、916=x-height）与 OS/2 表里的 `sCapHeight/sxHeight` 呼应，是 hinting 把字形关键高度“吸”到整数像素的依据。

6.15 可变字体表族 (fvar / STAT / avar / gvar / HVAR)

可变字体 (Variable Font) 把一个**设计空间**（如字重从细到粗）压缩进单一文件：默认实例是基线轮廓，其余位置由各字形的“差量(delta)”经插值得到。相关表分工如下，演示字体 `NotoSerifCJKsc-vf.ttf`（思源宋体可变版）齐全——注意它是 **TrueType(glyf)** 轮廓的可变字体，而静态的思源宋体是 CFF(.otf)：

表	作用
<code>fvar</code>	定义 轴 （轴标签 / 最小值 / 默认值 / 最大值）与若干 命名实例 （预设快捷档）
<code>STAT</code>	给每个轴值取名并标注取值范围，供应用的“字重选择器”展示
<code>avar</code>	把用户输入的轴坐标（归一化后） 非线性重映射 到设计的归一化值
<code>gvar</code>	每个字形在每个轴方向的 点位移差量 （决定形状如何随轴变化）
<code>HVAR/VVAR</code>	水平 / 垂直 推进宽度 (advance) 的差量（决定间距如何随轴变化）

这个可变字体比静态字体多了 `fvar/STAT/avar/gvar/HVAR` 五张表。`ttx -l` 节选这几张及相关表如下（`cmap/GSUB` 等其余表与静态字体无异，已省略）：

```

1 $ ttx -l NotoSerifCJKsc-VF.ttf
2     tag      checksum      length      offset
3     ----      -
4     HVAR  0xD4DA58D3      63253  35578604
5     STAT  0x80E79632       182  35641860
6     avar  0x79F58926       42  35642044
7     fvar  0x8D4C75DC       106  35642088
8     glyf  0xCCA2ECA7  34269540  759092
9     gvar  0x032BB0A0  24257464  35642196  ← 24 MB, 全字体最大

```

`fvar` / `STAT` / `avar` 都很小（百字节级），而 `gvar`（24 MB）甚至比 `glyf`（34 MB）以外的任何表都大——它承载了全部 65535 个字形在轴上的差量。

6.15.1 `fvar`：轴定义与命名实例

```

1 <fvar>
2   <!-- weight -->
3   <Axis>
4     <AxisTag>wght</AxisTag>
5     <Flags>0x0</Flags>
6     <MinValue>200.0</MinValue>
7     <DefaultValue>200.0</DefaultValue>
8     <MaxValue>900.0</MaxValue>
9     <AxisNameID>265</AxisNameID>      <!-- 用 name 表 265 号字符串命名此轴 -->
10  ->
11  </Axis>
12
13  <!-- 7 个命名实例: ExtraLight/Light/Regular/Medium/SemiBold/Bold/Black -->
14  <NamedInstance flags="0x0" postscriptNameID="267" subfamilyNameID="266">
15    <coord axis="wght" value="200.0"/>    <!-- ExtraLight -->
16  </NamedInstance>
17  ...
18  <NamedInstance flags="0x0" postscriptNameID="271" subfamilyNameID="270">
19    <coord axis="wght" value="400.0"/>    <!-- Regular -->
20  </NamedInstance>
21  ...
22 </fvar>

```

- 单轴 `wght`，范围 200–900。默认值 = 200（ExtraLight）——即此字体的“原点”是最细款，只有“加粗”方向有差量。
- 7 个 `NamedInstance` 是预置快捷档（下拉菜单里的选项），坐标如 `value="400.0"`；名字存在 `name` 表，由 `subfamilyNameID` 引用。

6.15.2 `STAT`：样式属性轴值表

`fvar` 只点了几个离散实例，`STAT` 则给整条轴的连续区间命名，让应用既列预设、又能自由滑动：

```

1 <STAT>
2   <DesignAxisRecordSize value="8"/>
3   <DesignAxisRecord>
4     <Axis index="0"><AxisTag value="wght"/><AxisNameID value="265"/>
5   <AxisOrdering value="0"/></Axis>
6 </DesignAxisRecord>

```

```

6      <!-- AxisValueCount=7 -->
7      <AxisSizeArray>
8          <AxisSize index="2" Format="2">
9              <AxisIndex value="0"/>
10             <Flags value="2"/>          <!-- ELIDABLE: 常规字重，菜单中可省略 -->
11         >
12             <ValueNameID value="260"/>      <!-- Regular -->
13             <NominalValue value="400.0"/>
14             <RangeMinValue value="350.0"/>
15             <RangeMaxValue value="499.0"/>
16         </AxisSize>
17         ... (共 7 个，覆盖 200/300/400/500/600/700/900) ...
18     </AxisSizeArray>
19     <ElidedFallbackNameID value="256"/>    <!-- 省略时回退显示 Regular -->
20 </STAT>

```

- `Format="2"` 是带区间的轴值（标称值 + Min/Max 范围）。Regular 的 `Flags value="2"`（ELIDABLE）表示它可被省略——常规字重不显示在字重菜单里。

6.15.3 avar：轴的非线性映射

没有 `avar` 时插值是纯线性的；`avar` 用分段映射把用户输入的归一化值(`from`)重映射到设计的归一化值(`to`)，实现非均匀的字重曲线：

```

1  <avar>
2    <version major="1" minor="0"/>
3    <segment axis="wght">
4      <mapping from="-1.0" to="-1.0"/>
5      <mapping from="0.0" to="0.0"/>
6      <mapping from="0.1429" to="0.095"/>
7      <mapping from="0.2857" to="0.21"/>
8      <mapping from="0.4286" to="0.36"/>
9      <mapping from="0.5714" to="0.51"/>
10     <mapping from="0.7143" to="0.73"/>
11     <mapping from="1.0" to="1.0"/>
12   </segment>
13 </avar>

```

- 归一化：默认值(200)→0.0，最大值(900)→1.0。7 个命名实例的归一化位置恰好是 0.0，0.1429，0.2857，0.4286，0.5714，0.7143，1.0——正是 `from` 列。
- `to` 列把中段"压"向细端（如 0.4286→0.36 而非 0.4286），即字重视觉上**先慢后快**地变粗，让各档字重的视觉间距更均匀。

6.15.4 gvar：字形差量数据

`gvar` 为每个字形记录各轴方向的**点位移差量**。字形 `A`（29 个点）在 `wght` 轴峰值 1.0（最粗）处的差量：

```

1 <glyphVariations glyph="A">
2   <tuple>
3     <coord axis="wght" value="1.0"/>      <!-- 本 tuple 在 wght=1.0(=900, 最
粗) 处取峰值 -->
4     <delta pt="0" x="3" y="0"/>
5     <delta pt="1" x="3" y="5"/>
6     <delta pt="2" x="5" y="10"/>
7     ...
8     <delta pt="8" x="67" y="20"/>          <!-- 该点右移 67、上移 20 -->
9     <delta pt="9" x="77" y="0"/>          <!-- 横画右端点外扩 77 (加粗主因) -->
10    ...
11    <delta pt="28" x="0" y="0"/>
12  </tuple>
13 </glyphVariations>

```

- 把 glyph 里 A 每个点的坐标加上对应 (x,y) 差量, 即得到 Bold 版的 A; 中间轴位置由两端按 `avar` 曲线插值。
- `<delta pt="9" x="77" y="0"/>` 这种大位移对应横画加粗后端点外扩。一个 `<tuple>` 对应一个"主样式"(master); 本字体只有默认(200)+最粗(900)两个 master, 故大多数字形 (65022 个) 各含一个 tuple——另有 513 个零差量字形 (如空格, 加粗也不变形)。

6.15.5 HVAR: 水平度量差量

`gvar` 只管字形形状, **水平推进宽度**(advance width) 的变化由 `HVAR` 提供, 否则加粗后字间距会全错。它用 `ItemVariationStore` (区域 + 差量表) + **间接映射** 做去重压缩:

```

1 <HVAR>
2   <Version value="0x00010000"/>
3   <VarStore Format="1">
4     <VarRegionList>
5       <!-- RegionCount=1 -->
6       <Region index="0">
7         <VarRegionAxis index="0">
8           <StartCoord value="0.0"/><PeakCoord value="1.0"/><EndCoord
value="1.0"/>
9         </VarRegionAxis>
10        </Region>
11      </VarRegionList>
12      <!-- VarDataCount=2 -->
13      <VarData index="0">
14        <!-- ItemCount=89 -->
15        <VarRegionIndex index="0" value="0"/>
16        <Item index="0" value="[-101]"/>
17        <Item index="4" value="[0]"/>
18        ...
19      </VarData>
20      <VarData index="1">
21        <!-- ItemCount=13 -->
22        <Item index="0" value="[133]"/>
23        ...
24      </VarData>
25    </VarStore>
26    <AdvWidthMap>

```

```

27     <Map glyph=".notdef"  outer="0" inner="4"/>
28     <Map glyph="A"        outer="0" inner="40"/>
29     <Map glyph="quotedbl" outer="1" inner="1"/>
30     ...
31 </AdvWidthMap>
32 </HVAR>

```

- 差量被高度去重：65535 个字形只存了 $89+13 = 102$ 个**唯一宽度差量**（`VarData` 的 `Item`），每个字形用 `AdvWidthMap` 的 `outer` (选哪个 `VarData`) + `inner` (选哪一项) 间接引用。如 `A` → `VarData[0]` 第 40 项，`quotedbl` → `VarData[1]` 第 1 项。
- 正因如此 `HVAR` 才 63 KB——若每个字形各存一份将是数 MB。竖排度量的差量由 `VVAR` 承担（本字体无此表）。

dump 这些表的技巧：`gvar` 全量 dump 会生成上百 MB 的 XML。检视单个字形时，用 `fontTools` 的 Python API 只取一个字形即可（下例输出即为真实 TTX 片段）：

```

1 from fontTools.ttLib import TTFont
2 font = TTFont("NotoSerifCJKsc-VF.ttf")
3 gvar = font["gvar"]
4 gvar.variations = {"A": gvar.variations["A"]} # 只留 A
5 font.saveXML("A_gvar.ttx", tables=["gvar"])

```

第七部分 字体特性与功能

7.1 两种轮廓格式

	TrueType glyf	PostScript CFF
曲线	二次贝塞尔（on/off 点）	三次贝塞尔
数据形态	显式点坐标 + 指令	紧凑的 charstring 操作码
hinting	指令式（强大、可编程）	声明式（BlueValues 等）
典型	<code>.ttf</code> （如 Times、雅黑）	<code>.otf</code> （如思源、Adobe 字体）

判定方法：`ttx -l` 看有 `glyf` 还是 `CFF` 表。

7.2 TTC 字体集合

`.ttc`（TrueType Collection）/ `.otc` 把多个**共享字形数据**的字体打包在一个文件里（如一套字体的多个字重），省空间。用 `ttx -y N` 选择第 `N` 个（从 0 起）。本文中 `msyh.ttc`、`cambria.ttc` 各含 2 个字体。

7.3 OpenType 布局特性（GSUB/GPOS）

让字体支持复杂排版：连字（liga）、小型大写（smcp）、花体替换（swsh）、字距（kern）、阿拉伯文/天城文等复杂脚本的形变。通过 GSUB（替换）+ GPOS（定位）实现，按 脚本→语言→特性→查找 四级组织。

7.4 数学排版 (MATH 表)

仅数学字体 (如 Cambria Math) 拥有。提供数学常量 (上下标缩放、轴高、极限堆叠间距)、拉伸字形 (巨型括号/根号)、斜体修正等, 是 Word 公式、LaTeX 引擎高质量渲染数学公式的基础。

7.5 CJK 纵向排版

中文/日文/韩文需要竖排, 靠 `vhea/vmtx` (纵向度量) + `VORG` (CFF 纵向原点) + `BASE` (表意字基线) + `GSUB` 里的竖排异体替换 (如标点旋转) 实现。雅黑、思源都有完整的纵向表。

7.6 TrueType hinting

TrueType 字体可在小字号下用字节码指令把字形对齐到像素网格, 保证清晰锐利。三张表配合: `fpgm` (函数)、`prep` (预程序)、`cvt` (控制值)。ttx 默认反汇编成可读指令, `-i` 可保留为 hex。

7.7 嵌入式点阵 (EBDT/EBLC、CBDT/CBLC)

部分字体在小字号下用预渲染的位图替代矢量轮廓以保证清晰 (如 Cambria 的 `EBDT/EBLC`)。ttx 用 `-z` 控制位图导出格式 (`raw/row/bitwise/extfile`), 其中 `bitwise` 能肉眼看到点阵形状。

7.8 可变字体 (fvar/STAT/avar/gvar/HVAR)

可变字体 (Variable Fonts) 在一份文件里用 `fvar` (轴定义, 如字重/字宽)、`STAT` (轴值命名与取值范围)、`avar` (用户轴→设计轴的非线性映射)、`gvar` (字形差量)、`HVAR/VVAR` (水平/垂直度量差量) 表达一个连续设计空间, 浏览器/应用通过插值得到任意中间样式, 无需为每个字重单独打包字体。本文演示字体 `NotoSerifCJKsc-VF.ttf` (思源宋体可变版) 是一个单 `wght` 轴 (200-900) 的可变字体, 各表的 TTX 结构详见 [6.15 节](#)。ttx 能正常 dump/compile 这些表; 编译时加 `--optimize-font-speed` 会改变 `gvar/VARC` 的编码方式, 产物更大但 HarfBuzz 等渲染更快。

第八部分 常见 workflow

8.1 修改字体名称

```
1 ttx -t name font.ttf # 只导出 name 表
2 # 编辑 font.ttx 里的 <namerecord>
3 ttx -m font.ttf -o newfont.ttf font.ttx # 合并回原字体
```

8.2 提取 / 替换单张表

```
1 ttx -t cmap font.ttf # 提取 cmap
2 # 修改后
3 ttx -m font.ttf -o fixed.ttf font.ttx
```

8.3 字体子集化

ttx 本身不做子集化, 但同属 fontTools 的 `pyftsubset` 是配套工具, 可按字符集裁剪字体 (Web 嵌入常用):

```
1 pyftsubset font.ttf --text="Hello世界" --output-file=font.subset.ttf
```

8.4 对比两个字体

```
1 ttx -x glyf -o a.ttf fontA.ttf
2 ttx -x glyf -o b.ttf fontB.ttf
3 diff a.ttf b.ttf # 排除轮廓后对比元数据差异
```

8.5 转换为 WOFF2

```
1 ttx font.ttf # 字体 → ttx
2 ttx --flavor woff2 -o font.woff2 font.ttf # ttx → woff2
```

8.6 检视 / 修改可变字体的轴

先确认是不是可变字体（含 `fvar` / `gvar`），再单独导出轴定义表做检视或修改：

```
1 ttx -l font.ttf # 看是否含 fvar/gvar（可变字体）
2 ttx -t fvar -t STAT -t avar -t name font.ttf # 只导出轴定义与命名
3 # 编辑 font.ttf 里的轴范围 / 命名实例后，合并回原字体
4 ttx -m font.ttf -o newfont.ttf font.ttf
```

要从一个可变字体提取某个静态字重（如 Bold 700），用同属 fontTools 的实例化器（`fontTools.varLib.instancer`，Python API）：

```
1 python -c "from fontTools.ttLib import TTFont; from fontTools.varLib import
    instancer; \
2 instancer.instantiateVariableFont(TTFont('font.ttf'),
    {'wght':700}).save('font-Bold.ttf')"
```

实例化产物不再含 `fvar` / `gvar` / `HVAR` 等表，是一个普通静态字体（`ttx -l` 可验证）。

附录 A 命令行选项速查表

选项	类别	作用
<code>-h</code>	通用	打印帮助
<code>--version</code>	通用	显示版本
<code>-d <目录></code>	通用	输出目录（须先存在）
<code>-o <文件></code>	通用	输出文件名； <code>-o -</code> 为 stdout（与 <code>-d</code> 同用时优先）
<code>-f</code>	通用	覆盖而非追加序号
<code>-v</code>	通用	详细输出
<code>-q</code>	通用	静默
<code>-a</code>	通用	允许虚拟字形 ID
<code>-l</code>	Dump	列表模式（不导出）

选项	类别	作用
<code>-t <表></code>	Dump	只导出指定表（可多个）
<code>-x <表></code>	Dump	排除指定表（可多个，与 <code>-t</code> 互斥）
<code>-s</code>	Dump	按表拆分
<code>-g</code>	Dump	按字形拆分 glyf（隐含 <code>-s</code> ）
<code>-i</code>	Dump	指令不反汇编（输出 hex）
<code>-z <格式></code>	Dump	位图格式 raw/row/bitwise/extfile
<code>-e</code>	Dump	不忽略错误（完整 traceback）
<code>-y <编号></code>	Dump	选 TTC 字体编号
<code>--unicodedata</code>	Dump	自定义字符名数据库
<code>--newline</code>	Dump	行尾 LF/CR/CRLF
<code>-m <字体></code>	Compile	合并 TTX 进已有字体
<code>-b</code>	Compile	不重算字形 bbox
<code>--recalc-timestamp</code>	Compile	时间戳设为当前
<code>--no-recalc-timestamp</code>	Compile	保留原时间戳
<code>--flavor <类型></code>	Compile	woff / woff2
<code>--with-zopfli</code>	Compile	WOFF 用 Zopfli 压缩
<code>--optimize-font-speed</code>	Compile	渲染速度优先

附录 B 演示字体清单

文件	类型	来源	演示重点
<code>times.ttf</code>	西文 TrueType	Windows 系统字体	glyf 轮廓、TrueType hinting(fpgm/prep/cvt)、 GSUB/GPOS/JSTF、hdmx/VDMX
<code>msyh.ttc</code>	中文 TTC(2 字体)	Windows 微软雅黑	TTC 集合(<code>-y</code>)、超大 cmap、CJK 纵向 排版(vhea/vmtx)、MERG
<code>cambria.ttc</code>	数学 TTC(2 字体)	Windows Office	MATH 表 、嵌入式点阵 (EBDT/EBLC)、 <code>-z</code> 位图格式
<code>SourceHanSansSC- Regular.otf</code>	中文 CFF	思源黑体 (开源)	CID-keyed CFF 、VORG、BASE、6 字 节 maxp、超大 GSUB

文件	类型	来源	演示重点
NotoSerifCJKsc-VF.ttf	中文可变 TrueType	思源宋体可变版(开源)	可变字体： fvar/STAT/avar/gvar/HVAR，单 wght 轴(200–900)

以上均为真实字体（系统自带或开源），非合成。

附录 C 常见表标签速查

标签	全称	作用
head	Font Header	字体头（版本、unitsPerEm、bbox、时间戳）
name	Naming	字体名称、版权、版本字符串
hhea	Horizontal Header	水平排版头
hmtx	Horizontal Metrics	每字形水平度量（width/lsb）
vhea	Vertical Header	纵向排版头（CJK 竖排）
vmtx	Vertical Metrics	每字形纵向度量
maxp	Maximum Profile	字形数 + 限制（TTF 详细 / CFF 极简）
os/2	OS/2 and Windows Metrics	字重、字宽、PANOSE、Win 度量
post	PostScript Name	字形名、斜体角、下划线
cmap	Character to Glyph Mapping	Unicode → 字形
loca	Index to Location	glyph 的字形偏移索引
glyf	Glyph Data	TrueType 轮廓
CFF	Compact Font Format	PostScript/CFF 轮廓（含 CID）
VORG	Vertical Origin	CFF 纵向原点
BASE	Baseline Data	多脚本基线
GSUB	Glyph Substitution	替换（连字/小型大写等）
GPOS	Glyph Positioning	定位（字距/标注）
GDEF	Glyph Definition	字形类别、附着点
JSTF	Justification	两端对齐
fpgm	Font Program	TrueType 全局函数
prep	Control Value Program	TrueType 预程序

标签	全称	作用
cvt	Control Value Table	TrueType hinting 控制值
gasp	Grid-fitting/Scan-conversion	按字号的 hinting 策略
MATH	Math Layout	数学排版常量与变体
EBDT / EBLC	Embedded Bitmap Data/Location	灰度点阵
CBDT / CBLC	Color Bitmap Data/Location	彩色点阵
hdmx	Horizontal Device Metrics	按字号的水平度量
VDMX	Vertical Device Metrics	按字号的垂直度量
LTSH	Linear Threshold	线性缩放阈值
kern	Kerning	旧式字距（已被 GPOS 取代）
DSIG	Digital Signature	数字签名
meta	Metadata	元数据
fvar	Font Variations	可变字体轴定义
STAT	Style Attributes	可变字体轴值命名与取值范围（供 UI）
avar	Axis Variations	可变字体轴的非线性映射
gvar	Glyph Variations	可变字体字形变体
HVAR / VVAR	Metrics Variations	可变字体度量变体

文档生成环境：fontTools 4.61.0 · Windows 11 · 演示命令均实际运行验证。